

QuantAI V2

Business Requirements & Technical Architecture Document

Project Name: QuantAI Automated Trading Platform

Document Type: Detailed BRD & Functional Specs

Version: 2.0 (Post Live News Sentiment Integration)

Date: 15 June 2026

Confidentiality: Strictly Confidential

TABLE OF CONTENTS

1. Executive Summary

1.1 Project Overview

1.2 Business Objectives

1.3 Scope of Version 1 & 2

2. System Architecture

2.1 High-Level Design (HLD)

2.2 Technology Stack Matrix

3. Functional Requirements: Backend & AI Engine

3.1 Market Data Ingestion Pipeline

3.2 Quantitative Technical Indicators

3.3 Machine Learning (AI) Predictive Engine

3.4 V2 Live Macro Sentiment Analyzer

3.5 Automated Order Execution Engine

4. Functional Requirements: Frontend Dashboard

4.1 UI/UX Design Language

4.2 Real-time Dashboard Components

5. Offline Backtesting Engine

5.1 Core Architecture

5.2 Metrics & Reporting

6. Non-Functional Requirements

6.1 Performance & Latency Constraints

6.2 Security & Risk Management

1. EXECUTIVE SUMMARY

1.1 Project Overview

QuantAI is a state-of-the-art, fully autonomous algorithmic trading platform specifically engineered for the Indian Financial Markets (NSE/BSE). The system is designed to eliminate human emotion from intraday trading by systematically executing trades in index options (NIFTY & BANKNIFTY). It achieves this by aggregating complex mathematical price-action indicators, predictive Machine Learning models, and real-time Natural Language Processing (NLP) of macroeconomic news.

1.2 Business Objectives

- **Zero-Touch Automation:** Operate a fully hands-free trading lifecycle—from market data polling to signal generation, order execution, and position exit via dynamic trailing stop-losses.
- **Statistical Edge:** Ensure every trade has a statistically proven positive expectancy by combining quantitative technicals with AI probability scoring.
- **Risk Mitigation:** Enforce strict institutional-grade risk management protocols to protect capital against sudden market crashes or black-swan events.

1.3 Scope of Versions (V1 & V2)

Version 1 (V1) established the core infrastructure: the Fyers API trading bridge, the technical indicator engine (CPR, Squeeze, EMA), the Random Forest ML model, the historical backtesting engine, and the Next.js React frontend dashboard.

Version 2 (V2) introduced the Live News Sentiment Analyzer. This critical subsystem scrapes top financial RSS feeds (Economic Times, Livemint), scores them using the VADER NLP model to gauge market panic/euphoria, translates headlines into regional languages (Telugu) for user accessibility, and displays them via a non-blocking background architecture.

2. SYSTEM ARCHITECTURE

2.1 High-Level Design (HLD)

The QuantAI architecture is strictly decoupled into three operational layers to ensure low latency and high reliability:

1. **Presentation Layer (Next.js Frontend):** A client-side rendered dashboard communicating via REST APIs. It handles visualization without executing any trading logic.
2. **Application Layer (FastAPI Backend):** The core Uvicorn server. It runs multiple asynchronous background threads (e.g., the Trading Loop, the Sentiment Scraper, Model Retraining) to ensure the REST API remains non-blocking and highly responsive.
3. **Execution & Data Layer (Fyers API & local storage):** Handles order placement over secure WebSockets/REST, and maintains local JSON/CSV stores for tick data and AI model weights (Pickle files).

2.2 Technology Stack Matrix

Frontend (Presentation)

- **Framework:** Next.js 14 (App Router)
- **Language:** TypeScript
- **Styling:** TailwindCSS (Dark/Glassmorphism)
- **Charting:** Recharts, Framer Motion
- **Icons:** Lucide React

Backend (Application)

- **Framework:** FastAPI (Asynchronous)
- **Server:** Uvicorn
- **Language:** Python 3.x
- **Concurrency:** ThreadPoolExecutor, Daemon Threads

Data Science & AI (Intelligence)

- **Data Manipulation:** Pandas, NumPy
- **Machine Learning:** Scikit-Learn (Random Forest)
- **NLP / Text:** VADER Sentiment, deep-translator

Brokerage & Infrastructure

- **Broker API:** Fyers API V3
- **Data Feeds:** NSE Tick Data via Fyers WebSockets
- **News Feeds:** Feedparser (RSS XML)

3. FUNCTIONAL REQUIREMENTS: BACKEND & AI

3.1 Market Data Ingestion Pipeline

The system must reliably ingest high-frequency market data. It connects to the Fyers API to pull historical minute-level candles for initial state building, and subsequently subscribes to WebSocket streams (or high-frequency polling) for live tick data during market hours (09:15 AM to 03:30 PM IST).

3.2 Quantitative Technical Indicators Engine

The core rule-based logic relies on the continuous calculation of the following mathematical models:

INDICATOR	LOGIC & FUNCTIONAL REQUIREMENT
Central Pivot Range (CPR)	Calculates Top Central (TC), Pivot (P), and Bottom Central (BC) based on previous day's High, Low, and Close. Used strictly for defining institutional support/resistance zones.
Squeeze Momentum	Evaluates the relationship between Bollinger Bands and Keltner Channels. A "Squeeze Fired" signal indicates an imminent, explosive directional breakout.
Exponential Moving Averages	Tracks 9 EMA and 21 EMA. Crossovers are utilized as secondary confirmation signals for entry timing.
Volume Profile	Identifies Point of Control (POC) and Value Areas (VAH, VAL) by analyzing traded volume at specific price nodes, distinguishing retail noise from institutional buying.

3.3 Machine Learning (AI) Predictive Engine

Rule-based systems are prone to false breakouts. The AI engine acts as the primary filter.

Requirement: The system implements a `RandomForestClassifier`. It consumes a feature matrix derived from the indicators (e.g., `Distance_to_EMA`, `RSI_Value`, `CPR_Width`) and outputs a probability array: `[P(Call), P(Put), P(Neutral)]`. A trade is only authorized if the directional probability exceeds a high-confidence threshold (e.g., `> 65%`).

3.4 V2 Live Macro Sentiment Analyzer

Market technicals fail during sudden macroeconomic news events. V2 introduces a robust NLP pipeline to monitor real-world sentiment.

- Data Scraping:** Asynchronously polls RSS feeds from Top Tier financial publishers (Economic Times, Livemint, Moneycontrol) every 5 minutes.
- NLP Scoring:** Headlines are parsed through the `vaderSentiment` lexicon. Each headline receives a compound score. The aggregated score determines the Macro State: Panic, Bearish, Neutral, Bullish, or Euphoria.
- Localization:** Headlines are passed through `deep-translator` (Google Translate API) to generate Telugu equivalents (`title_te`), facilitating regional user comprehension on the UI.
- System Protection:** The backend process runs as a daemon thread (`_background_fetch`) ensuring the heavy NLP and network I/O operations do not block the primary `FastAPI Uvicorn` event loop.

3.5 Automated Order Execution Engine

Once a high-probability signal aligns with the macro sentiment, the execution engine places the trade.

- **Order Type:** Market Orders for immediate fill of Index Options (Options Chain parsing required).
- **Risk Mapping:** Every order is accompanied by a hard Stop Loss.
- **Trailing SL:** The engine continuously monitors open positions. If PnL crosses predefined milestones (e.g., +10%), the Stop Loss is automatically modified (trailed) to breakeven, and subsequently trailed higher to lock in profits.

4. FUNCTIONAL REQUIREMENTS: FRONTEND DASHBOARD

4.1 UI/UX Design Language

The frontend is designed to reflect an institutional, high-end "Quant" aesthetic. It strictly adheres to a Dark Mode palette (Slate/Gray backgrounds) infused with Glassmorphism (translucent panels, backdrop blur). Accent colors (Emerald Green for Bullish, Rose Red for Bearish, Sky Blue for Neutral) are used sparingly to draw attention to critical metrics.

4.2 Real-time Dashboard Components

- **AI Strategy Probabilities:** Visual circular or bar progress indicators showing the live probability of Call vs. Put scenarios.
 - **Macro Sentiment Meter:** A gauge or visual text block indicating the aggregated VADER NLP score of global news.
 - **Bilingual News Ticker:** A horizontal ticker component that automatically rotates through the top 5 parsed headlines every 4 seconds. Supports user-toggled translation (English to Telugu) and includes clickable outbound links to the original articles.
 - **Active Trades Panel:** A data grid displaying currently open positions, entry prices, live PnL, and the status of the Trailing Stop Loss.
-

5. OFFLINE BACKTESTING ENGINE

5.1 Core Architecture

Located in `backtesting_engine/run.py`, this module operates independently of the live broker. It ingests large CSV datasets of historical minute-data and processes them through the exact same indicator and AI logic used in production.

5.2 Metrics & Reporting

The backtester generates a comprehensive post-simulation report including:

METRIC	DEFINITION
Total Return / CAGR	The absolute percentage gain of the strategy over the backtest duration.
Maximum Drawdown (MDD)	The largest peak-to-trough drop in account equity. A strict threshold (e.g., < 15%) is required for strategy approval.
Win Rate & Profit Factor	Percentage of winning trades vs. losing trades, and Gross Profit divided by Gross Loss.
Trade Log Output	A detailed CSV output of every hypothetical execution for granular manual review.

6. NON-FUNCTIONAL REQUIREMENTS (NFR)

6.1 Performance & Latency Constraints

- Execution Latency:** The time elapsed from signal generation to order placement via Fyers API must not exceed 250 milliseconds.
- API Responsiveness:** Internal FastAPI endpoints (e.g., `/api/sentiment`, `/api/signals`) must respond within 50ms. Heavy tasks must be offloaded to `ThreadPoolExecutor`.
- Frontend Rendering:** The Next.js UI must maintain 60 FPS during chart animations and data polling updates.

6.2 Security & Risk Management

Critical Risk Protocols: The system must enforce a daily maximum loss limit. If the account equity drops by the defined daily threshold (e.g., 2% of capital), the TradingLoop must execute an emergency halt, close all open positions at market price, and reject any further signals until the next trading session.

- **API Key Protection:** Fyers API credentials, App IDs, and Secret Keys must be managed strictly via `.env` variables and never hardcoded into the source.
- **State Recovery:** In the event of a server crash or power failure, the bot must be able to read active order IDs from the broker upon reboot to accurately reconstruct its state without duplicating trades.

7. FUTURE ENHANCEMENTS ROADMAP (V3)

The following features are slated for the next major release cycle to further enhance the institutional capabilities of the platform:

Deep Learning Upgrade

Transitioning the predictive engine from Random Forest to LSTM (Long Short-Term Memory) neural networks to better capture sequential time-series patterns in price action.

Options Greeks Decoding

Real-time calculation of Delta, Theta, Vega, and Gamma. This will allow the bot to execute complex non-directional hedging strategies like Iron Condors and Straddles.

Multi-Broker Adapters

Abstracting the broker execution layer to seamlessly support Zerodha (Kite Connect), Upstox, and Angel One simultaneously for load balancing and redundancy.

Push Notifications

Integration with Telegram API and WhatsApp Business API to send instant alerts for trade executions, trailing stop updates, and daily PnL summaries.

-- End of Document --